

Unsupervised Financial Anomaly Detection via Isolation Forest from First Principles

Date: 24/09/2026

1. Objective

To implement an Isolation Forest anomaly detection algorithm from scratch without using a ready-made Isolation Forest library. The algorithm recursively partitions transaction data, builds multiple isolation trees, and calculates anomaly scores.

2. What is Anomaly Detection?

Anomaly detection is the process of finding unusual or suspicious data points. In financial transactions, anomalies may represent unusual spending, fraud, or abnormal transaction behavior.

3. Isolation Forest

Isolation Forest is an unsupervised machine learning algorithm. It works on the idea that anomalies are easier to isolate than normal observations. A random feature and a random split value are selected repeatedly. An anomalous transaction usually reaches an isolated leaf in fewer splits.

4. Main Steps

Step	Description
1	Select a random subset of transaction data.
2	Randomly choose a feature.
3	Choose a random split value between the minimum and maximum values.
4	Split the data into left and right groups.
5	Repeat recursively until the stopping condition is reached.
6	Build many isolation trees.
7	Calculate the average path length for every transaction.
8	Convert the average path length into an anomaly score.

5. Mathematical Anomaly Score

For a transaction x , let $E(h(x))$ be its average path length over all isolation trees. The anomaly score is calculated as:

$$s(x, n) = 2^{(-E(h(x)) / c(n))}$$

where:

$E(h(x))$ = average path length of x

n = sample size

$c(n)$ = average path length of an unsuccessful search in a binary search tree

$$c(n) = 2H(n-1) - 2(n-1)/n$$

$H(i)$ is the harmonic number.

A score closer to 1 indicates a stronger anomaly, while a lower score indicates a more normal observation.

6. Python Implementation from Scratch

```
import random
import math
import statistics

data = [
    [100, 2],
    [120, 3],
    [110, 2],
    [130, 4],
    [115, 3],
    [125, 2],
    [1000, 50], # anomaly
    [118, 3],
    [122, 4],
    [105, 2]
]

class Node:
    def __init__(self, left=None, right=None, split_att=None,
                  split_value=None, size=0):
        self.left = left
        self.right = right
        self.split_att = split_att
        self.split_value = split_value
        self.size = size

def c_factor(n):
    if n <= 1:
        return 0
    h = sum(1 / i for i in range(1, n))
    return 2 * h - 2 * (n - 1) / n

def build_tree(data, height, max_height):
    if height >= max_height or len(data) <= 1:
        return Node(size=len(data))

    n_features = len(data[0])
    feature = random.randrange(n_features)

    values = [row[feature] for row in data]
    min_value = min(values)
    max_value = max(values)

    if min_value == max_value:
        return Node(size=len(data))

    split = random.uniform(min_value, max_value)

    left_data = [row for row in data if row[feature] < split]
    right_data = [row for row in data if row[feature] >= split]

    if not left_data or not right_data:
        return Node(size=len(data))

    left = build_tree(left_data, height + 1, max_height)
    right = build_tree(right_data, height + 1, max_height)

    return Node(left, right, feature, split, len(data))

def path_length(point, node, current_height=0):
    if node.left is None or node.right is None:
        return current_height + c_factor(node.size)

    if point[node.split_att] < node.split_value:
        return path_length(point, node.left, current_height + 1)
    else:
        return path_length(point, node.right, current_height + 1)

def isolation_forest(data, n_trees=50, sample_size=8):
    trees = []
    max_height = math.ceil(math.log2(sample_size))

    for _ in range(n_trees):
        sample = random.sample(data, min(sample_size, len(data)))
        tree = build_tree(sample, 0, max_height)
        trees.append(tree)

    scores = []

    for point in data:
        paths = [path_length(point, tree) for tree in trees]
        average_path = statistics.mean(paths)
```

```
        score = 2 ** (-average_path / c_factor(sample_size))
        scores.append(score)

    return scores

scores = isolation_forest(data)

for transaction, score in zip(data, scores):
    print(transaction, "Anomaly Score:", round(score, 4))
```

7. Expected Interpretation

The transaction [1000, 50] is very different from the other transactions. It is therefore expected to receive a relatively high anomaly score. The remaining transactions are closer to the normal pattern and should receive lower scores.

8. Advantages

- Works without labelled fraud/anomaly data.
- Can handle multiple transaction attributes.
- Anomalies are isolated quickly.
- Does not require distance calculations between all data points.
- Useful for large financial transaction datasets.

9. Conclusion

Isolation Forest is an effective unsupervised technique for detecting unusual financial transactions. By randomly partitioning data and measuring how quickly observations are isolated, the algorithm can identify potential anomalies without requiring labelled examples.